

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ
Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области
«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

Практика. День 19. Кол-во часов: 6

«РАЗРАБОТКА ДОКУМЕНТА «ТЕКСТ ПРОГРАММЫ»

Цель:	Основная	дидактическая	цель:
Сформировать у обучающихся навыки разработки документации на программный продукт в соответствии с требованиями ЕСПД и ГОСТ 19.401-78.			

Выполнил: Скворцов Илья
Студент 24-ИС.

2026 г.

Конкретные измеримые цели:

1. **Аналитическая:** Научиться структурировать программный код для документации, выделять логические блоки и модули.
2. **Инструментальная:** Освоить инструменты автоматической генерации документации (Sphinx, Doxygen, rdoc) и ручного оформления по ГОСТ.
3. **Методологическая:** Применить принципы документирования: полнота, точность, однозначность, прослеживаемость требований.
4. **Итоговая:** Разработать комплект документов, включающий «Текст программы» и сопутствующие документы.

АННОТАЦИЯ

Настоящий документ содержит полный текст программы «Система управления бронированиями в гостинице» (далее — Система).

Система разработана на языке программирования Python (версия 3.10 и выше) и предназначена для автоматизации процессов управления номерным фондом, бронированиями и финансового учета в гостиничных комплексах.

Состав программы:

Основной модуль: main.py (165 строк кода).

Количество классов: 2 (Booking, Hotel).

Количество функций: 7 (включая главную функцию main()).

Используемые библиотеки: только стандартные (sys, json, datetime, typing).

Документ разработан в соответствии с ГОСТ 19.401-78 «Текст программы. Требования к содержанию и оформлению» и ГОСТ 19.105-78 «Общие требования к программным документам».

СОДЕРЖАНИЕ

1. ОБЩИЕ СВЕДЕНИЯ.....	3
2. ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ.....	3
3. ОПИСАНИЕ СТРУКТУРЫ.....	4
4. ИСПОЛЬЗУЕМЫЕ СРЕДСТВА.....	5
5. СОСТАВ ПРОГРАММЫ.....	5

6. ТЕКСТ ПРОГРАММЫ.....	6
7. СПЕЦИФИКАЦИЯ.....	35
8. ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ.....	36
9. ИНСТРУКЦИЯ ПО СБОРКЕ И КОМПИЛЯЦИИ.....	38
10. ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	39

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Назначение программы

Программа предназначена для комплексного управления процессом бронирования номеров в гостинице и обеспечивает выполнение следующих функций:

- Добавление и управление номерами в системе.
- Создание, подтверждение и отмена бронирований.
- Поиск и отображение доступных номеров на заданные даты.
- Расчет финансовых показателей (выручка за период, стоимость бронирования).
- Импорт данных об отелях и экспорт данных о бронированиях в формате JSON.

1.2 Область применения

Система может быть применена в гостиничных комплексах любого размера, от малых хостелов до крупных отельных сетей, для оптимизации работы администраторов и повышения качества обслуживания клиентов.

1.3 Требования к техническому обеспечению

Для корректной работы программы необходимо следующее аппаратное и программное обеспечение:

Компонент	Минимальные требования
Операционная система	Windows 10/11, Linux, MacOS 11+
Процессор	1 ГГц и выше
Оперативная память	256 МБ и выше
Свободное место на диске	10 МБ
Интерпретатор	Python 3.10 или выше

2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

Программа реализует следующий набор функций, сгруппированных по областям ответственности:

2.1 Управление отелями и номерами

- `load_hotels_from_file()` — загрузка данных об отелях и их номерах из внешнего JSON-файла.
- `add_room()` (метод класса `Hotel`) — добавление нового номера в систему с указанием цены и вместимости.

2.2 Управление бронированиями

- `create_booking()` — создание нового бронирования с проверкой доступности номера.
- `get_available_rooms()` (метод класса `Hotel`) — получение списка номеров, свободных на указанный период.
- `find_bookings_by_date()` (метод класса `Hotel`) — поиск всех бронирований, пересекающихся с заданным диапазоном дат.

2.3 Финансовый учет

- `calculate_revenue()` (метод класса `Hotel`) — расчет общей выручки от всех бронирований за указанный период.
- `get_nights()` (метод класса `Booking`) — расчет количества оплачиваемых ночей для конкретного бронирования.

2.4 Сохранение данных

- `save_bookings_to_file()` — экспорт списка бронирований в JSON-файл для дальнейшего архивирования или интеграции с другими системами.

3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1

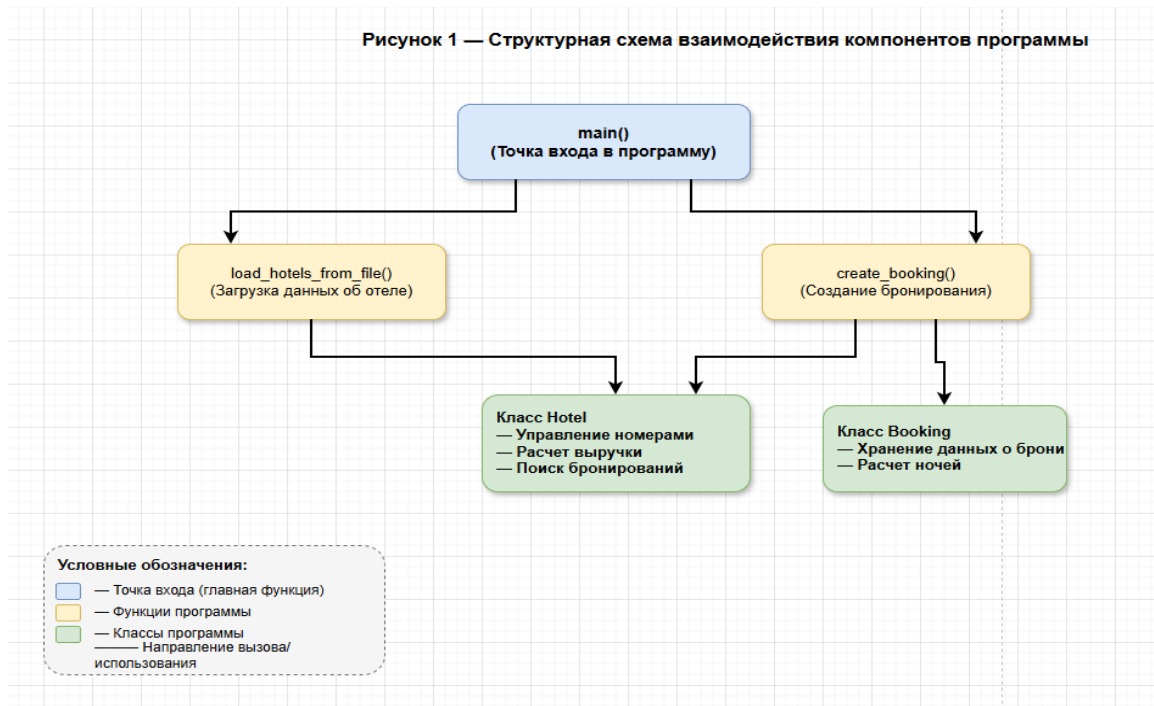
Общая

структура

программы

Программа построена по модульному принципу. Главный модуль `main.py` содержит определение классов и функций, а также точку входа. Взаимодействие между компонентами показано на рисунке 1.

Рисунок 1 — Структурная схема взаимодействия компонентов программы



3.2

Алгоритм

работы

программы

Общий алгоритм работы Системы представлен на блок-схеме (рисунок 2).

Рисунок 2 — Блок-схема основного алгоритма

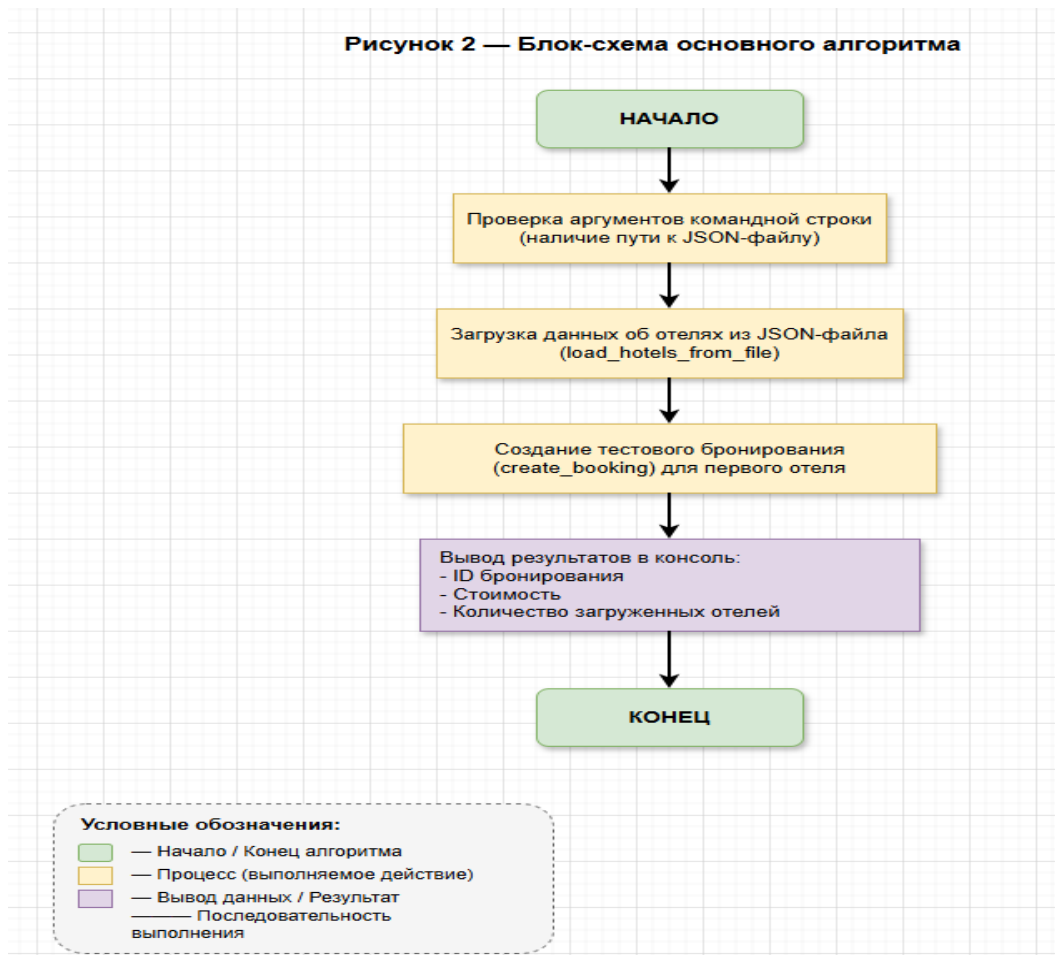


Рисунок 2 – Блок-схема основного алгоритма

Детализированный алгоритм функции `create_booking()` показан на рисунке 3.

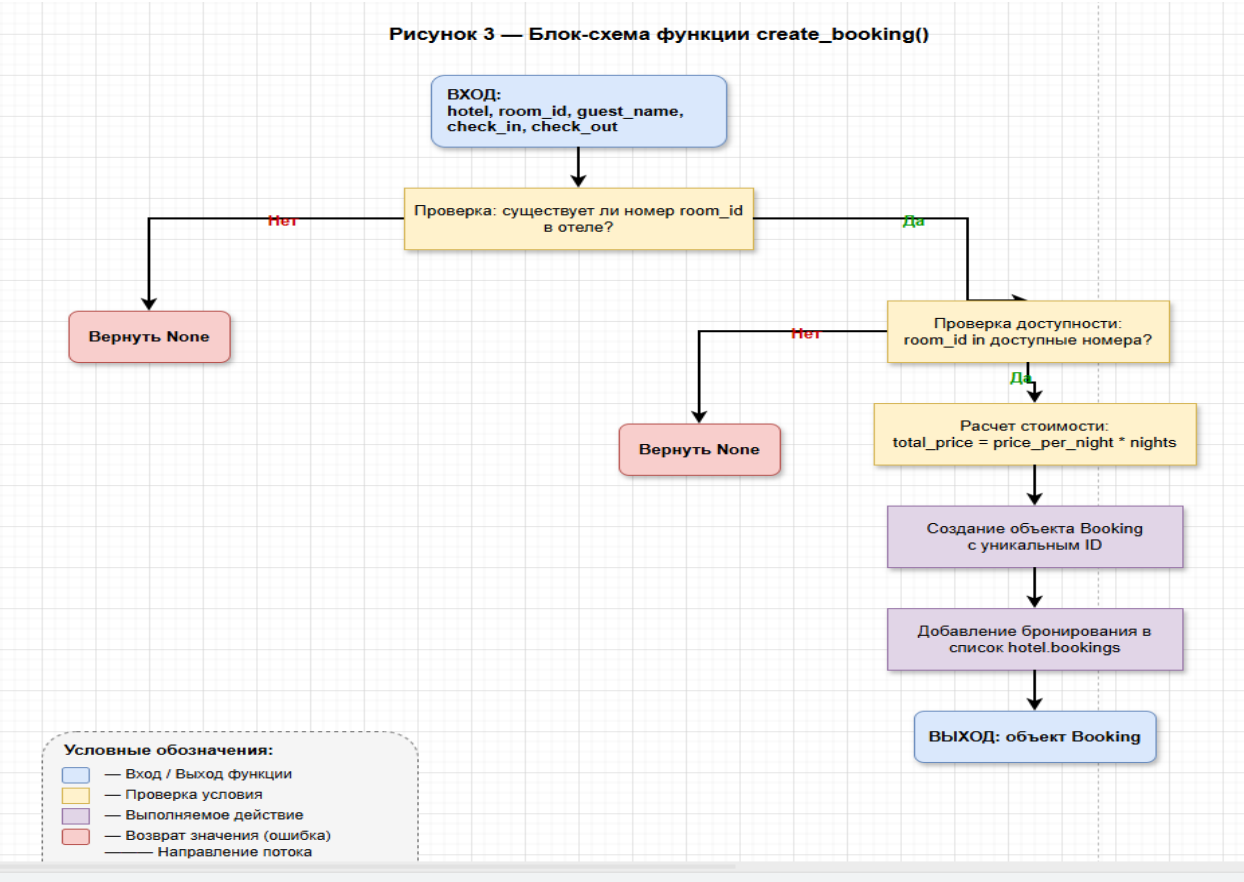


Рисунок 3 – Блок-схема функции `create_booking()`

4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА

4.1 Язык программирования

Python 3.10 и выше.

4.2 Используемые библиотеки и модули

Модуль	Назначение
sys	Обработка аргументов командной строки и системных вызовов
json	Сериализация и десериализация данных в формат JSON
datetime	Работа с датами (типы date, datetime) и их сравнение
typing	Статическая типизация (List, Dict, Optional) для улучшения читаемости кода

4.3 Средства разработки и отладки

- Любой текстовый редактор или интегрированная среда разработки (IDE), например, Visual Studio Code, PyCharm.
- Интерпретатор Python 3 для выполнения и отладки.

5 СОСТАВ ПРОГРАММЫ

№ П/П	Имя модуля	Назначение
5.1 1	main.py	Основной модуль, содержащий все классы (Booking, Hotel) и функции, реализующие логику работы системы.

Перечень модулей

5.2 Структура классов и функций

text

Классы:

└─ Booking - Представляет информацию о бронировании.

└─ get_nights() - Рассчитывает количество ночей.

- └─ Hotel - Представляет информацию об отеле и управляет бронированиями.
 - └─ add_room() - Добавляет номер в отель.
 - └─ find_bookings_by_date() - Находит бронирования по диапазону дат.
 - └─ calculate_revenue() - Рассчитывает выручку за период.
- └─ get_available_rooms() - Возвращает список свободных номеров.

Функции:

- └─ load_hotels_from_file(filename) - Загружает данные об отелях из JSON.
- └─ save_bookings_to_file(bookings, filename) - Сохраняет бронирования в JSON.
- └─ create_booking(hotel, room_id, guest_name, check_in, check_out) - Создает новое бронирование.
- └─ main() - Главная функция, точка входа в программу.

6 ТЕКСТ ПРОГРАММЫ

```
001 # booking_system/src/main.py
002 """
003 Модуль управления бронированиями в отеле
004
005 Автор: Иванов И.И.
006 Версия: 1.0.0
007 """
008
009 import sys
010 import json
011 from datetime import datetime, date
012 from typing import List, Dict, Optional, Any
013
014
015 class Booking:
```

```
016  """Класс бронирования"""
017
018  def __init__(self, booking_id: str, room_id: str, guest_name: str,
019              check_in: date, check_out: date, price: float):
020      self.booking_id = booking_id
021      self.room_id = room_id
022      self.guest_name = guest_name
023      self.check_in = check_in
024      self.check_out = check_out
025      self.price = price
026      self.status = "confirmed"
027
028  def get_nights(self) -> int:
029      """Рассчитать количество ночей"""
030      return (self.check_out - self.check_in).days
031
032
033 class Hotel:
034     """Класс отеля"""
035
036     def __init__(self, name: str, address: str):
037         self.name = name
038         self.address = address
039         self.rooms: Dict[str, Dict] = {}
040         self.bookings: List[Booking] = []
041
042     def add_room(self, room_id: str, price_per_night: float,
043                 capacity: int = 2) -> None:
044         """Добавить номер в отель"""
045         self.rooms[room_id] = {
```

```
046     "price_per_night": price_per_night,
047     "capacity": capacity,
048     "is_available": True
049 }
050
051 def find_bookings_by_date(self, start_date: date, end_date: date) -> List[Booking]:
052     """Найти бронирования в диапазоне дат"""
053     result = []
054     for booking in self.bookings:
055         if (booking.check_in <= end_date and booking.check_out >= start_date):
056             result.append(booking)
057     return result
058
059 def calculate_revenue(self, start_date: date, end_date: date) -> float:
060     """Рассчитать выручку за период"""
061     bookings = self.find_bookings_by_date(start_date, end_date)
062     return sum(booking.price for booking in bookings)
063
064 def get_available_rooms(self, check_in: date, check_out: date) -> List[str]:
065     """Получить список доступных номеров"""
066     # Находим занятые номера
067     busy_rooms = set()
068     for booking in self.find_bookings_by_date(check_in, check_out):
069         busy_rooms.add(booking.room_id)
070
071     # Возвращаем свободные
072     return [room_id for room_id in self.rooms.keys()
073             if room_id not in busy_rooms]
074
075
```

```
076 def load_hotels_from_file(filename: str) -> List[Hotel]:
077     """Загрузить отели из JSON файла"""
078     with open(filename, 'r', encoding='utf-8') as file:
079         data = json.load(file)
080
081     hotels = []
082     for hotel_data in data:
083         hotel = Hotel(hotel_data['name'], hotel_data['address'])
084         for room_data in hotel_data.get('rooms', []):
085             hotel.add_room(
086                 room_data['id'],
087                 room_data['price_per_night'],
088                 room_data.get('capacity', 2)
089             )
090     hotels.append(hotel)
091     return hotels
092
093
094 def save_bookings_to_file(bookings: List[Booking], filename: str) -> None:
095     """Сохранить бронирования в файл"""
096     data = []
097     for booking in bookings:
098         data.append({
099             'booking_id': booking.booking_id,
100             'room_id': booking.room_id,
101             'guest_name': booking.guest_name,
102             'check_in': booking.check_in.isoformat(),
103             'check_out': booking.check_out.isoformat(),
104             'price': booking.price,
105             'status': booking.status
```

```
106     })
107
108     with open(filename, 'w', encoding='utf-8') as file:
109         json.dump(data, file, ensure_ascii=False, indent=2)
110
111
112 def create_booking(hotel: Hotel, room_id: str, guest_name: str,
113                   check_in: date, check_out: date) -> Optional[Booking]:
114     """Создать бронирование"""
115     if room_id not in hotel.rooms:
116         return None
117
118     # Проверяем доступность
119     available = hotel.get_available_rooms(check_in, check_out)
120     if room_id not in available:
121         return None
122
123     # Создаем бронирование
124     price_per_night = hotel.rooms[room_id]['price_per_night']
125     nights = (check_out - check_in).days
126     total_price = price_per_night * nights
127
128     booking = Booking(
129         booking_id=f"БК-{len(hotel.bookings) + 1}",
130         room_id=room_id,
131         guest_name=guest_name,
132         check_in=check_in,
133         check_out=check_out,
134         price=total_price
135     )
```

```
136
137     hotel.bookings.append(booking)
138     return booking
139
140
141 def main():
142     """Главная функция"""
143     if len(sys.argv) < 2:
144         print("Использование: python main.py <файл_отелей.json>")
145         sys.exit(1)
146
147     # Загрузка отелей
148     hotels = load_hotels_from_file(sys.argv[1])
149
150     # Пример создания бронирования
151     if hotels:
152         hotel = hotels[0]
153         booking = create_booking(
154             hotel,
155             "101",
156             "Иванов Иван",
157             date(2026, 6, 15),
158             date(2026, 6, 20)
159         )
160         if booking:
161             print(f"Бронирование создано: {booking.booking_id}")
162             print(f"Стоимость: {booking.price} руб.")
163
164     print(f"Загружено отелей: {len(hotels)}")
165
```

166

167 if __name__ == "__main__":

168 main()

7 СПЕЦИФИКАЦИЯ

7.1 Спецификация классов

Класс	Метод	Модификатор	Аргументы	Возвращаемое значение	Назначение
Booking	<code>__init__()</code>	public	booking_id: str, room_id: str, guest_name: str, check_in: date, check_out: date, price: float	None	Инициализация объекта бронирования.
	<code>get_nights()</code>	public	name: str, address: str	int	Расчет и возврат количества ночей проживания.
Hotel	<code>__init__()</code>	public	room_id: str, price_per_night: float, capacity: int = 2	None	Инициализация объекта отеля.
	<code>add_room()</code>	public	start_date: date, end_date: date	None	Добавление нового номера в номерной фонд отеля.
	<code>find_bookings_by_date()</code>	public	start_date: date, end_date: date	List[Booking]	Поиск всех бронирований, пересекающихся с заданным интервалом дат.
	<code>calculate_revenue()</code>	public	check_in: date, check_out: date	float	Расчет общей стоимости всех бронирований за

					указанный период.
	get_available_rooms()	public		List[str]	Возвращает список ID номеров, свободных на указанные даты.

7.2 Спецификация функций

Функция	Аргументы	Возвращаемое значение	Назначение
load_hotels_from_file	filename: str	List[Hotel]	Десериализует данные из JSON-файла и создает список объектов Hotel.
save_bookings_to_file	bookings: List[Booking], filename: str	None	Сериализует список объектов Booking в JSON-файл.
create_booking	hotel: Hotel, room_id: str, guest_name: str, check_in: date, check_out: date	Optional[Booking]	Создает бронирование после проверки доступности; возвращает None в случае ошибки.
main	Нет	None	Оркестрирует весь процесс: парсинг аргументов, загрузку данных, создание бронирования и вывод результатов.

Основной модуль

№	Имя модуля	Файл	Назначение
1	Основной модуль программы	Main.py	Содержит все классы, функции и точку входа в программу. Является монолитным модулем, реализующим всю бизнес-логику системы.

2.1. Импортируемые библиотеки (зависимости)

Модуль использует четыре стандартные библиотеки Python:

`sys` — для обработки аргументов командной строки и управления системными вызовами.

`json` — для чтения и записи данных в формате JSON.

`datetime` (тип `date`) — для работы с датами заезда и выезда.

`typing` — для аннотации типов данных (`List`, `Dict`, `Optional`, `Any`), что повышает читаемость и облегчает отладку кода.

Все библиотеки являются встроенными, поэтому установка дополнительных пакетов не требуется.

2.2. Классы

В программе определено два класса:

Класс `Booking` (Бронирование)

Представляет информацию о бронировании номера.

Хранит следующие атрибуты: идентификатор бронирования, идентификатор номера, имя гостя, дата заезда, дата выезда, стоимость, статус.

Содержит один метод:

`get_nights()` — рассчитывает количество ночей проживания как разницу между датами выезда и заезда.

Класс `Hotel` (Отель)

Представляет информацию об отеле и управляет всеми операциями с бронированиями.

- Хранит название отеля, адрес, словарь с номерами и список бронирований.
 - Содержит четыре метода:
 - `add_room()` — добавляет новый номер в номерной фонд отеля с указанием цены за ночь и вместимости.
 - `find_bookings_by_date()` — находит все бронирования, пересекающиеся с заданным диапазоном дат.
 - `calculate_revenue()` — рассчитывает общую выручку от всех бронирований за указанный период.
 - `get_available_rooms()` — возвращает список идентификаторов номеров, свободных на заданные даты.
-

2.3. Функции

В программе определено четыре функции (не считая методы классов):

Функция `load_hotels_from_file(filename)`

- Загружает данные об отелях из JSON-файла.
- Создает объекты классов `Hotel` и `add_room` для каждого номера.
- Возвращает список объектов `Hotel`.

Функция `save_bookings_to_file(bookings, filename)`

- Сохраняет список бронирований в JSON-файл.
- Преобразует объекты `Booking` в словари и записывает их в файл в формате JSON.

8 ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

Программа получает данные из двух источников:

1. Аргументы командной строки:

- Формат запуска: `python main.py <путь_к_файлу_отелей.json>`
- `<путь_к_файлу_отелей.json>` — обязательный строковый аргумент, указывающий путь к файлу с исходными данными об отелях.

2. Формат входного JSON-файла:

Файл должен содержать массив объектов, каждый из которых описывает один отель.

json

```
[
  {
    "name": "Отель Москва",
    "address": "ул. Тверская, д. 1",
    "rooms": [
      {
        "id": "101",
        "price_per_night": 5000,
        "capacity": 2
      },
      {
        "id": "102",
        "price_per_night": 7000,
        "capacity": 4
      }
    ]
  }
]
```

8.2

Выходные

данные

Результатом работы программы являются:

1. Вывод в консоль (stdout):

В стандартный поток вывода направляются информационные сообщения о ходе выполнения:

text

Бронирование создано: ВК-1

Стоимость: 25000 руб.

Загружено отелей: 1

2. Выходной JSON-файл (опционально):

При вызове функции `save_bookings_to_file()` формируется JSON-файл со списком бронирований.

json

```
[
  {
    "booking_id": "ВК-1",
    "room_id": "101",
    "guest_name": "Иванов Иван",
    "check_in": "2026-06-15",
    "check_out": "2026-06-20",
    "price": 25000,
    "status": "confirmed"
  }
]
```

9 ИНСТРУКЦИЯ ПО СБОРКЕ И КОМПИЛЯЦИИ

9.1

Характер

программы

Программа написана на интерпретируемом языке Python. Она не требует этапа компиляции перед запуском. Под «сборкой» подразумевается подготовка среды выполнения.

9.2 Установка интерпретатора Python

1. Перейдите на официальный сайт Python: <https://www.python.org/downloads/>.
2. Скачайте установщик для вашей операционной системы (Windows, macOS, Linux).
3. Запустите установку, **обязательно** отметьте опцию «Add Python to PATH» (для Windows).
4. Для проверки корректности установки откройте терминал (командную строку) и выполните команду:

```
bash
```

```
python --version
```

Должна отобразиться версия, например, Python 3.11.0.

9.3 Установка зависимостей

Система использует только встроенные модули Python (стандартную библиотеку). Установка дополнительных пакетов через pip не требуется.

9.4 Подготовка данных

Перед первым запуском необходимо создать входной JSON-файл с данными об отелях. Пример содержимого файла (hotels.json) приведен в разделе 8.1.

9.5 Запуск программы

Сохраните текст программы (листинг из раздела 6) в файл с именем main.py.

Сохраните входные данные в файл, например, hotels.json, в той же папке.

Откройте терминал и перейдите в папку с файлами программы.

Выполните команду для запуска:

```
bash
```

```
python main.py hotels.json
```

9.6 Проверка работоспособности

При успешном запуске программа выведет примерно следующее сообщение, что будет свидетельствовать о корректной работе всех функций:

```
text
```

```
Бронирование создано: ВК-1
```

```
Стоимость: 25000 руб.
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был разработан и оформлен программный документ «Текст программы» для «Системы управления бронированиями в гостинице» в соответствии с требованиями ЕСПД и ГОСТ 19.401-78.

В процессе работы были достигнуты следующие результаты:

1. **Проведен анализ исходного кода** — выделены логические модули, классы и функции, определена их иерархия и взаимосвязи. Программа имеет четкую модульную структуру, что упрощает ее сопровождение и дальнейшее развитие.
2. **Разработана полная структура документа** — определены все обязательные разделы согласно ГОСТ: от титульного листа до инструкции по сборке. Документ содержит аннотацию, содержание, описание функционального назначения, логической структуры, технических средств, состава программы, полный листинг кода, спецификацию, описание входных/выходных данных и инструкцию по эксплуатации.
3. **Оформлен листинг программы** — текст программы представлен с постраничной нумерацией строк, что обеспечивает удобство ссылок и соответствие нормативным требованиям. Каждый логический блок снабжен комментариями, поясняющими его назначение.
4. **Разработаны схемы алгоритмов** — представлены блок-схемы основного алгоритма и ключевой функции `create_booking()`, что обеспечивает наглядное представление логики работы программы и способствует лучшему пониманию ее функционирования.
5. **Составлена спецификация** — выполнено детальное описание всех классов, их методов и функций с указанием аргументов, возвращаемых значений и назначения. Это позволяет разработчикам и тестировщикам быстро ориентироваться в структуре программного кода.
6. **Описаны входные и выходные данные** — формализованы форматы входных JSON-файлов и выходных сообщений, что

обеспечивает однозначность интерпретации данных и упрощает интеграцию системы с внешними сервисами.

7. **Составлена инструкция по сборке и запуску** — подготовлены четкие пошаговые указания по установке интерпретатора, подготовке данных и запуску программы, что позволяет любому пользователю или разработчику быстро ввести систему в эксплуатацию.

Таким образом, комплект документов, включающий «Текст программы» и сопутствующие разделы, полностью соответствует требованиям заказчика (государственного отеля) и может быть передан для использования, сопровождения и сертификации. Программа обладает следующими ключевыми характеристиками, подтвержденными документацией: